



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E WEB PROGRAMMING AND INTERFACE

JAVASCRIPT EVENTS

- JavaScript events are actions or occurrences that happen in the browser, which can be detected and responded to using JavaScript code. Events can be triggered by user interactions (like clicks or key presses) or by the browser itself (like page loading).

TYPES OF EVENTS

- User Interaction Events:
- Mouse Events:
 - click: Triggered when an element is clicked.
 - dblclick: Triggered on a double-click.
 - mouseover: Triggered when the mouse pointer moves over an element.
 - mouseout: Triggered when the mouse pointer leaves an element.

TYPES OF EVENTS

- User Interaction Events:
- Keyboard Events:
 - keydown: Triggered when a key is pressed down.
 - keyup: Triggered when a key is released.
 - keypress: Triggered when a key is pressed (deprecated in favor of keydown and keyup).
- Form Events:
 - submit: Triggered when a form is submitted.
 - change: Triggered when the value of an input changes.

TYPES OF EVENTS

- Browser Events:
- load: Triggered when the entire page (including images) is loaded.
- resize: Triggered when the browser window is resized.
- scroll: Triggered when the document is scrolled.

TYPES OF EVENTS

- Touch Events:
- For touch-enabled devices:
 - touchstart: Triggered when a finger touches the screen.
 - touchmove: Triggered when a finger moves on the screen.
 - touchend: Triggered when a finger is removed from the screen.

EVENT HANDLING

- An event listener is a function that waits for a specific event to occur on a specified element.

EVENT HANDLING

- Adding an Event Listener

```
const button =  
document.getElementById("myButton");  
  
button.addEventListener("click", function() {  
    alert("Button was clicked!");  
});
```

EVENT HANDLING

- Removing an Event Listener
- You can also remove an event listener if needed:

```
function handleClick() {  
    alert("Button was clicked!");  
}  
  
button.addEventListener("click", handleClick);  
  
// Later on, to remove the listener:  
button.removeEventListener("click", handleClick);
```

THE EVENT OBJECT

- When an event occurs, an event object is created, which contains useful information about the event. You can access it in your event handler.

```
button.addEventListener("click", function(event) {  
    console.log(event); // Log the event object  
    console.log(event.target); // The element that  
    triggered the event  
});
```

EVENT DELEGATION

- Technique where attach a single event listener to a parent element instead of multiple listeners on child elements.
- More efficient, especially for dynamically created elements.

EVENT DELEGATION

```
const list = document.getElementById("myList");

list.addEventListener("click", function(event) {
  if (event.target.tagName === "LI") {
    alert(`You clicked on
    ${event.target.textContent}` );
  }
});
```

EXAMPLE: FORM VALIDATION

- Events in a form validation:

```
<form id="myForm">
```

```
  <input type="text" id="name" placeholder="Enter  
your name" required />
```

```
  <button type="submit">Submit</button>
```

```
</form>
```

```
<script>
```

```
const form = document.getElementById("myForm");
```

EXAMPLE: FORM VALIDATION

```
form.addEventListener("submit", function(event) {  
    const nameInput =  
document.getElementById("name");  
    if (!nameInput.value) {  
        alert("Name is required!");  
        event.preventDefault(); // Prevent form  
submission  
    }  
});  
</script>
```

EXAMPLE: FORM VALIDATION

```
form.addEventListener("submit", function(event) {  
    const nameInput =  
document.getElementById("name");  
    if (!nameInput.value) {  
        alert("Name is required!");  
        event.preventDefault(); // Prevent form  
submission  
    }  
});  
</script>
```

STRINGS IN JAVASCRIPT

- A string is a sequence of characters used to represent text.
- Strings are immutable, meaning they cannot be changed once created.

CREATING STRINGS

- Using Quotes:
 - Single quotes: 'Hello'
 - Double quotes: "Hello"
 - Backticks (Template Literals): `Hello`

```
const str1 = 'Hello';
```

```
const str2 = "World";
```

```
const str3 = `Hello, ${str2}!`; // Template literal
```

STRING PROPERTIES

- Length:
- The length property returns the number of characters in a string.

```
const myString = "Hello";  
console.log(myString.length); // 5
```

COMMON STRING METHODS

- Basic Methods:
 - `toLowerCase()`: Converts a string to lowercase.
 - `toUpperCase()`: Converts a string to uppercase.
 - `trim()`: Removes whitespace from both ends of a string.

```
const message = " Hello, World! ";  
console.log(message.trim()); // "Hello, World!"
```

SEARCHING IN STRINGS

- IndexOf:
- Returns the index of the first occurrence of a specified value. Returns -1 if not found.

```
const str = "Hello, World!";  
console.log(str.indexOf("World")); // 7
```

SEARCHING IN STRINGS

- Includes:
- Checks if a string contains a specified substring. Returns true or false.

```
console.log(str.includes("Hello")); // true
```

SEARCHING IN STRINGS

- `StartsWith / EndsWith`:
- `startsWith()`: Checks if a string starts with a specified substring.
- `endsWith()`: Checks if a string ends with a specified substring.

```
console.log(str.startsWith("Hello")); // true  
console.log(str.endsWith("!")); // true
```

STRING REPLACEMENT

- Replace:
- Replaces the first occurrence of a substring with a new substring.

```
const newStr = str.replace("World", "JavaScript");  
console.log(newStr); // "Hello, JavaScript!"
```

STRING REPLACEMENT

- Replace All (ES2021):
- To replace all occurrences, you can use a regular expression with the g flag.

```
const text = "banana";  
const newText = text.replace(/a/g, "o");  
console.log(newText); // "bonono"
```

STRING SPLITTING AND JOINING

- Split:
- Splits a string into an array of substrings based on a specified delimiter.

```
const csv = "apple,banana,cherry";  
const fruits = csv.split(",");  
console.log(fruits); // ["apple", "banana", "cherry"]
```

STRING SPLITTING AND JOINING

- Join:
- Joins the elements of an array into a single string, with a specified separator.

```
const joined = fruits.join(" and ");  
console.log(joined); // "apple and banana and  
cherry"
```

TEMPLATE LITERALS

- Template literals are string literals allowing embedded expressions.
- They use backticks (`) instead of quotes.
- Multi-line Strings:
 - Template literals can span multiple lines.

```
const multiLine = `This is line one.  
This is line two.`;  
console.log(multiLine);
```

TEMPLATE LITERALS

- String Interpolation:
- You can embed variables and expressions using `${expression}`.

```
const name = "Alice";  
const greeting = `Hello, ${name}!`;   
console.log(greeting); // "Hello, Alice!"
```

TAGGED TEMPLATE LITERALS

- Tagged templates allow you to parse template literals with a function.

```
function tag(strings, ...values) {  
  console.log(strings); // Array of string parts  
  console.log(values); // Array of values  
  return `${strings[0]}${values[0]}!`; // Custom formatting  
}  
  
const name = "Bob";  
const result = tag`Hello, ${name}`;  
console.log(result); // "Hello, Bob!"
```

TAGGED TEMPLATE LITERALS

- Tagged templates allow you to parse template literals with a function.

```
function tag(strings, ...values) {  
  console.log(strings); // Array of string parts  
  console.log(values); // Array of values  
  return `${strings[0]}${values[0]}!`; // Custom formatting  
}  
  
const name = "Bob";  
const result = tag`Hello, ${name}`;  
console.log(result); // "Hello, Bob!"
```

NUMBERS IN JAVASCRIPT

- In JavaScript, numbers are represented as floating-point values.
- Can represent both integers and decimals.

NUMBER PROPERTIES

- Global Properties:
 - Number.MAX_VALUE: The largest positive finite value.
 - Number.MIN_VALUE: The smallest positive finite value.
 - Number.NEGATIVE_INFINITY: Represents negative infinity.
 - Number.POSITIVE_INFINITY: Represents positive infinity.
 - Number.NaN: Represents a "Not-a-Number" value.

```
console.log(Number.MAX_VALUE); // 1.7976931348623157e+308
```

```
console.log(Number.MIN_VALUE); // 5e-324
```

CREATING NUMBERS

- Number Literals:
 - Decimal: 42
 - Hexadecimal: 0x2A
 - Binary: 0b101010
 - Octal: 0o52
- Using the Number Constructor:

```
const num1 = Number("42"); // Converts string to number
```

```
const num2 = new Number(42); // Creates a Number object (not recommended)
```

NUMBER METHODS

- Common Methods:
- `Number.isInteger(value)`: Checks if the value is an integer.
- `Number.isNaN(value)`: Checks if the value is NaN.
- `Number.isFinite(value)`: Checks if the value is a finite number.

```
console.log(Number.isInteger(42)); // true  
console.log(Number.isNaN(NaN)); // true  
console.log(Number.isFinite(3.14)); // true
```

NUMBER FORMATTING

- `toFixed()`:
- Formats a number using fixed-point notation.

```
const num = 3.14159;  
console.log(num.toFixed(2)); // "3.14"
```

NUMBER FORMATTING

- `toFixed()`:
- Formats a number to a specified length.

```
console.log(num.toFixed(3)); // "3.14"
```

NUMBER FORMATTING

- `toExponential()`:
- Returns a string representing the number in exponential notation.

```
console.log(num.toExponential(2)); // "3.14e+0"
```

BIGINT IN JAVASCRIPT

- BigInt is a built-in object that provides a way to represent whole numbers larger than $2^{53} - 1$, which is the largest safe integer in JavaScript.
- Creating BigInts:
- Using the BigInt constructor:

```
const bigInt1 = BigInt(123456789012345678901234567890);
```

BIGINT IN JAVASCRIPT

- Using the n suffix:

```
const bigInt2 = 123456789012345678901234567890n;
```

BIGINT PROPERTIES AND METHODS

- Common Properties:
- BigInts can be used with most arithmetic operators.
- They cannot be mixed with regular numbers in arithmetic without explicit conversion.
- Methods:
- Conversion:
 - Convert BigInt to Number: `Number(bigInt)`
 - Convert Number to BigInt: `BigInt(num)`

```
const bigIntValue = 123456789012345678901234567890n;
```

```
const numberValue = Number(bigIntValue); // Loss of precision possible
```

ARITHMETIC WITH NUMBERS AND BIGINTS

- Operations:
- BigInt supports addition, subtraction, multiplication, and division.

```
const a = 12345678901234567890n;
```

```
const b = 2n;
```

```
const sum = a + b; // 12345678901234567892n
```

LIMITATIONS AND CONSIDERATIONS

- Mixed Types:
 - You cannot directly mix BigInt and Number types in expressions.

```
const num = 5;  
const bigInt = 10n;  
// This will throw an error  
// const result = num + bigInt;
```

DATE IN JAVASCRIPT

- The Date object in JavaScript represents dates and times.
- It is used for date manipulation and calculation.

CREATING DATE OBJECTS

- Using the Date Constructor:
 - Without arguments: Creates a date object for the current date and time.

```
const now = new Date();
```

CREATING DATE OBJECTS

- With Specific Date and Time:
 - You can specify year, month (0-11), day, hours, minutes, seconds, and milliseconds.

```
const specificDate = new Date(2023, 10, 3, 15, 30, 0); // Nov 3, 2023, 15:30:00
```

- From a Date String:

```
const dateFromString = new Date("2023-11-03T15:30:00");
```

DATE FORMATS

- ISO 8601 Format:
 - YYYY-MM-DDTHH:mm:ss.sssZ
 - Example: 2023-11-03T15:30:00Z
- Custom Formats:
 - You can format dates to display in various formats using methods like `toLocaleDateString()`.

COMMON DATE METHODS

- Getting Date Components:
 - `getFullYear()`: Returns the year (4 digits).
 - `getMonth()`: Returns the month (0-11).
 - `getDate()`: Returns the day of the month (1-31).
 - `getDay()`: Returns the day of the week (0-6).

COMMON DATE METHODS

```
const date = new Date();  
console.log(date.getFullYear()); // e.g., 2023  
console.log(date.getMonth()); // e.g., 10 for November  
console.log(date.getDate()); // e.g., 3  
console.log(date.getDay()); // e.g., 5 for Friday
```

SETTING DATE COMPONENTS

- Set Methods:
 - `setFullYear(year)`: Sets the year.
 - `setMonth(month)`: Sets the month (0-11).
 - `setDate(day)`: Sets the day of the month (1-31).
 - `setHours(hours)`: Sets the hour (0-23).
 - `setMinutes(minutes)`: Sets the minutes (0-59).
 - `setSeconds(seconds)`: Sets the seconds (0-59).

SETTING DATE COMPONENTS

- `const futureDate = new Date();`
- `futureDate.setFullYear(2025);`
- `futureDate.setMonth(0); // January`
- `futureDate.setDate(1); // 1st`
- `console.log(futureDate); // Outputs date set to January 1, 2025`

DATE ARITHMETIC

- Date Calculations:
- Add or subtract days by adjusting milliseconds.

```
const tomorrow = new Date(now.getTime() + (24 * 60 * 60 * 1000)); //  
Add 1 day  
console.log(tomorrow);
```

DATE ARITHMETIC

- Time in Milliseconds:
- Use `getTime()` to get the time value in milliseconds since January 1, 1970.

```
const now = new Date();
```

```
const timestamp = now.getTime(); // Current timestamp
```

FORMATTING DATES

- `toLocaleDateString()`:
 - Converts the date to a locale-specific string.

```
const options = { year: 'numeric', month: 'long', day: 'numeric' };  
console.log(now.toLocaleDateString('en-US', options)); // e.g., "November  
3, 2023"
```

FORMATTING DATES

- `toISOString()`:
- Returns the date as a string in the ISO format.

```
console.log(now.toISOString()); // e.g., "2023-11-03T15:30:00.000Z"
```

TIMEZONE HANDLING

- UTC Methods:
- `getUTCFullYear()`, `getUTCMonth()`, etc., provide UTC time values.

```
console.log(now.getUTCFullYear()); // e.g., 2023
```

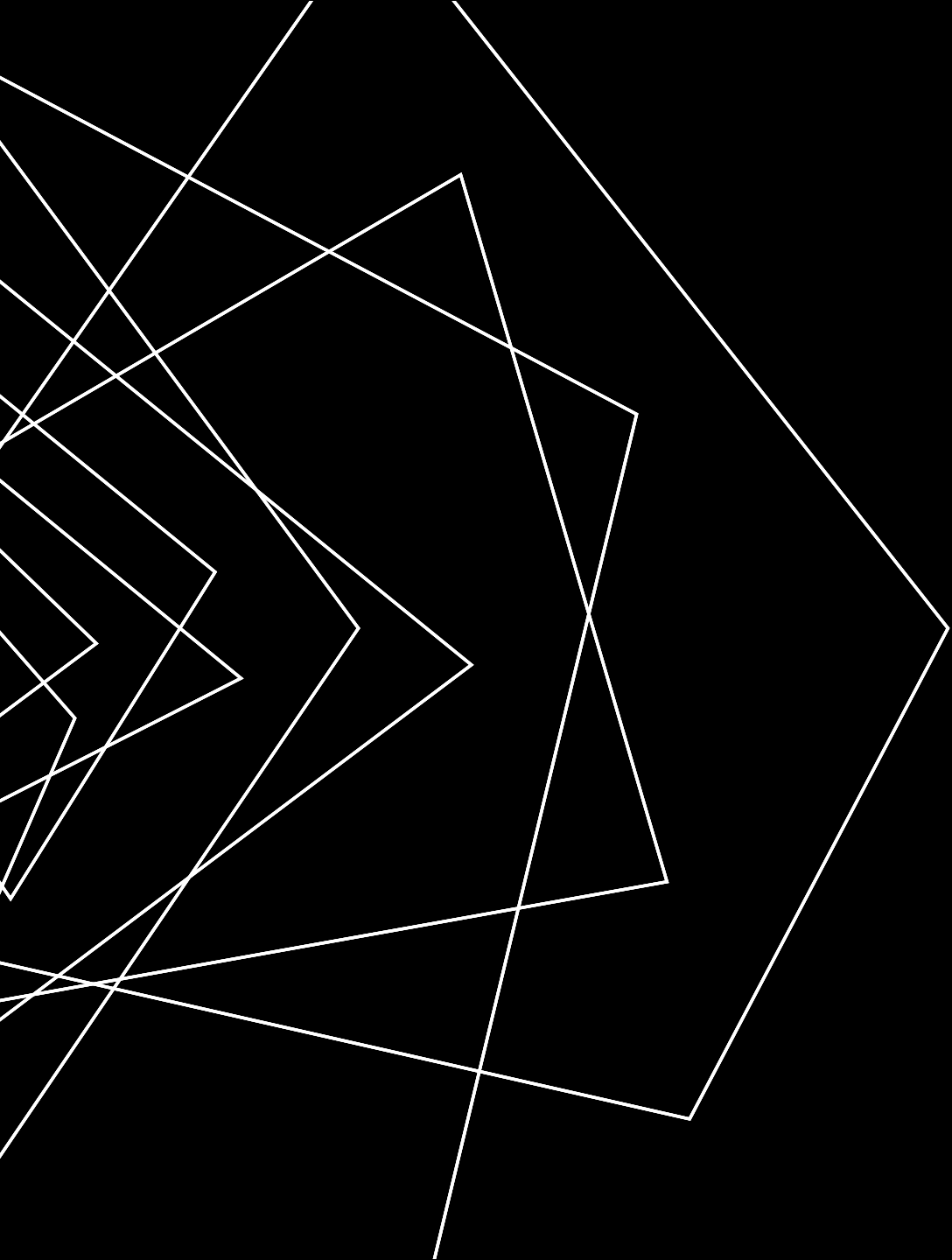
TIMEZONE HANDLING

- Setting Timezone Offsets:
- Use `toLocaleString()` with options to format dates in specific time zones.

```
const options = { timeZone: 'America/New_York' };  
console.log(now.toLocaleString('en-US', options));
```

COMMON PITFALLS

- Zero-Based Month Index:
 - Remember that months are zero-indexed (January is 0, December is 11).
- Date Parsing Issues:
 - Be cautious with string formats; use ISO format to avoid discrepancies.



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335